

Document Number:	P0056R00
Date:	2015-09-12
Project:	Programming Language C++, EWG
Revises:	none
Reply to:	gorn@microsoft.com

P0056R00: Soft Keywords

Bikeshed alternatives: scoped keywords, std-qualified keywords, named keywords.

Introduction

"We have to go with the odd ones, as all of the good ones are already taken". This was said a few times at different WG21 meetings. It was in a reference to keywords. C++ is a mature language with large existing codebases and an attempt to introduce a new keyword into the language will necessarily break existing code.

Quick sampling of some of the proposed keywords from Concepts, Modules, Transaction Memory, Pattern Matching, and Coroutines papers ([N3449](#), [N4134](#), [N4361](#), [N4466](#), [N4513](#), [PatternMatch]) in private and public codebases reveals that identifiers `await`, `concept`, `requires`, `synchronized`, `module`, `inspect`, `when` are used as names of variables, fields, parameters, namespaces and classes.

This paper explores the idea of adding *soft keywords* to the C++ language. This will enable new language features to select best possible keyword names without breaking existing software. The idea is simple. *Soft keyword* is a named entity implicitly defined in `std` or `std::experimental` namespaces that participates in the name lookup and name hiding according to existing language rules. If a name lookup finds an implicit declaration of a *soft keyword* it is treated in the same way as other context-dependent keyword resolved by the name lookup such as `typedef-name`, `namespace-name`, `class-name`, etc.

In the example below `yield` is a *soft keyword* implicitly defined in the `std` namespace.

```
namespace N1 { void yield(int); }
auto coro2() {
    using std::yield;
    yield(2);           // yields value 2
    N1::yield(3);       // invokes N1::yield
}
auto coro3() {
    using namespace N1;
    yield(1);           // invokes N1::yield
    std::yield(3);      // yields value 3
}
auto coro4() {
    using namespace N1;
    using namespace std;
    yield(4);           // error: ambiguous
}
```

Discussion

Drawback of the simple model described in the introduction is that without a using declaration or using directive, the developer need to always use `std::` with the *soft keyword*. This is troublesome as people would have to remember which keywords are the soft keywords and which are the good old "hard ones". This can be alleviated by adding a paragraph to the section 3.5 [basic.lookup] stating:

(5) if an unqualified name lookup [basic.lookup.unqual] or an argument-dependent name lookup [basic.lookup.argdep] fails to find a declaration and an identifier being looked up is a soft keyword identifier, it is treated as the corresponding context-dependent keyword

With this addition, we are getting to near *perfect* keyword experience. In the following example `module` is a soft keyword.

```
module A; // OK. Lookup did not find any

Xyz::Pcmf *module; // OK

bool FileHandleList::Find(LPCWSTR file)
{
    FileHandleCachePaths::iterator
        module = _Find(file); // OK
}
```

```
    return module != m_hInstPaths.end(); // OK
}
```

Dependent names

If a grammar construct utilizing a particular soft keyword can be interpreted as a function call when used in the template definition and being a dependent name, the current rules will result in the construct being treated as a function call. This preserves the property that a template can be correctly parsed prior to instantiation. That means that for some constructs, in templates, one must use explicitly qualified soft keywords, unless there a preceding using directive or declaration.

In the examples bellow, `inspect` and `when` are *soft keywords*.

```
template <typename T>
double area(const Shape& s, T u)
{
    inspect (s) { // OK: not a dependent name
        when Circle:    return 2*pi*radius();
        when Square:    return height()*width();
        default:        error("unknown shape");
    }
    std::inspect(u) { // must be qualified, otherwise will be parsed as a function call
        when X: return 1.0;
    }
}
```

Similarly, with `yield` soft keyword, in some cases, qualification will be needed.

```
template <typename T>
auto g() {
    T v;
    T* p;
    yield v;      // yield expression (not a dependent name, not a function call expr)
    yield(5);     // yield expression (not a dependent name)
    std::yield(v); // yield expression (not a dependent name, since qualified)
    std::yield *p; // yield expression (not a dependent name, since qualified)

    yield *p;     // operator * call, yield is not a soft keyword
    yield(v);     // function call, yield is a dependent name
}
```

This is unfortunate, but, developers are already trained to deal with two phase lookup in templates and take care of it, by inserting `typename`, `template` and `this->` as needed. Soft keywords add one more annoyance they have to deal with, unless we can take advantage of modules.

Modules to the rescue

However, situation is not as bleak as it may seem. Modules get us to perfect keyword experience, as they allow free use of using directives / declarations without exporting using directives / declarations outside of the module.

```
module A;

using namespace std;

template<typename T, typename U>
export void f(T& x, U xx)
{
    inspect (x,xx) { // OK: not a dependent name, as the lookup finds std::inspect
        when {int* p,0}:      p=nullptr;
        when {_a,int}:        ...    // _a is a placeholder matching everything
                                // shorthand for auto _a
    }
}
```

If someone finds using directive too broad, one can define a module with all of their favorite soft keywords exported in using declarations as follows:

```
module Cpp17keywords;
export {
    using std::inspect;
    using std::when;
```

```
using std::await;
using std::yield;
...
}
```

and now, any module can take advantage of using unqualified soft keywords by having `import Cpp17keywords;` declaration.

But it can still break some code

Yes. If a source file uses `using namespace std` and defines an entity with the name matching the soft keyword `xyz` in the global namespace or in another namespace `X` that is available for unqualified name lookup due to `using namespace X`, then, the lookup will be ambiguous. The fix would be to explicitly qualify the name in question with `::xyz` or `X::xyz`.

We can also do not break existing code, by altering paragraph 2 of section [namespace.udir] as follows (changes are in **bold**):

A *using-directive* specifies that the names in the nominated namespace can be used in the scope in which the *using-directive* appears after the *using-directive*. During unqualified name lookup (3.4.1), the names appear as if they were declared in the nearest enclosing namespace which contains both the using-directive and the nominated namespace. **This affects all names except the names of the soft keywords.**

One may ask, why should we do this? We don't guard against introducing new library functions in `std` namespace, why should we do this for keywords? For functions, library can rely on overloading and SFINAE to remove function names from consideration and reduce the chance of collision. We don't have this ability for keywords. Nevertheless, authors feel ambivalent about this rule and would like committee guidance.

Other concerns

What about tools? Would soft keywords confuse them? Not necessarily. If we introduce new constructs to the language tools need to be adapt to them.

Precise tools already have to rely on name lookup to figure out if `x * y;` is a declaration of a variable of type pointer to `x` or multiplication of `x` and `y`. Thus, they should be able to distinguish between identifiers and soft keywords.

Imprecise tools rely on heuristics to decide how to parse without having complete knowledge of all the symbols. In that case, they would have to use heuristics depending on the construct. For example, if `inspect(x)` is followed by the `{`, then heuristic would be that `inspect` is a keyword, otherwise, assume function name.

Implementation experience

A version of this proposal was implemented in non-shipping version of Microsoft C++ compiler.

Rough Wording

Here is a very rough sketch of how the wording might look for soft keywords. As an illustration, I use the soft keywords `yield` and `await`.

2.10 Identifiers

Add `yield` and `await` to the table 2 (Identifiers with special meaning).

3 Basic Concepts [basic]

In paragraph 3 add the text in **bold**.

An *entity* is a value, object, reference, function, enumerator, type, class member, template, template specialization, namespace, parameter pack, **soft keyword**, or `this`.

Add the following paragraph after paragraph 4.

If an unqualified name lookup [basic.lookup.unqual] or an argument-dependent name lookup [basic.lookup.argdep] fails to find a declaration and an identifier being looked up is a soft keyword identifier, it is treated as corresponding context-dependent keyword

3.4 Name lookup [basic.lookup]

In paragraph 1, add the text in bold.

The name lookup rules apply uniformly to all names (including *typedef-names* (7.1.3), *namespace-names* (7.3), ***soft-keyword-names*** (3.12), and *class-names* (9.1)) ...

3.12 Soft keywords

Soft keywords `yield` and `await` are implicitly declared in the `std::experimental` namespace. In the grammar productions, `yield-soft-keyword-name` and `await-soft-keyword-name` represent context-dependent keywords resulted from the name lookup according to the rules in 3.4 [basic.lookup].

5.3 Unary expressions

[Note: This is an illustration of soft keywords used in grammar production]

await-expression: `await-soft-keyword-name` *cast-expression*

References

N4134: Resumable Functions v2 (<http://www.open-std.org/JTC1/SC22/WG21/docs/papers/2014/n4134.pdf>)

N4361: C++ extensions for Concepts (<http://open-std.org/JTC1/SC22/WG21/docs/papers/2015/n4361.pdf>)

N3449: Open and Efficient Type Switch for C++ (<http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2012/n3449.pdf>)

N4466: Wording for Modules (<http://open-std.org/JTC1/SC22/WG21/docs/papers/2015/n4466.pdf>)

N4513: Working Draft Technical Specification for C++ Extensions for Transactional Memory (<http://www.open-std.org/JTC1/SC22/WG21/docs/papers/2015/n4513.pdf>)

[PatternMatch]: Presentation from the evening session at Urbana 2014